LABORATORY TITLE: FIREWALL TRAFFIC CONTROL AND FORENSIC VERIFICATION

STUDENT NAME: WALIYAT OLALEKAN

STUDENT ID: 2025/FWSD/11435

COURSE NAME: BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

COURSE CODE: SBT-DF203 (LAB 6)

INSTRUCTOR NAME: MR. AMINU IDRIS

DATE OF SUBMISSION: SEPTEMBER 18TH, 2026.

VERSION: Version 1.0

Executive Summary

This report documents the network forensic analysis of host-based firewall rules (iptables) executed on a Kali Linux environment running an Apache web server. The objective was to inspect the behavioral and forensic differences between allowed HTTP communication and firewall-suppressed connection attempts. Baseline captures established normal 3-way handshake and HTTP 200 OK transaction lifecycles. Subsequent application of an INPUT chain DROP rule demonstrated kernel-level packet silent discarding, resulting in client TCP [SYN] retransmissions, connection timeouts (curl error 28), and corresponding non-zero rule counter increments. All firewall modifications were verified, analyzed, and cleanly restored to baseline operational state.

Lab Network and Scope

- Target Workstation: Kali Linux (kali@kali)
- Network Interfaces: Loopback (lo - 127.0.0.1), Ethernet (eth0 - 192.168.40.128)
- Target Service: Apache2 Web Server listening on 0.0.0.0:80
- Scope: Local interface host-based traffic filtering, pcapng packet capture verification, hash integrity validation, and iptables rule state correlation

Original Firewall State and Evidence Hashes

- Initial Firewall Configuration: Uncomplicated Firewall (ufw) chains active with default policy.
- http_allowed.pcapng SHA-256: 88da6433a4b33e7f6d9234335ad8aeed632409560ebc92f05b6dda58d632a76a
- http_blocked.pcapng SHA-256: 64c595598401c8862fa4f215bd4a4025d7273686bb558e498e23be7d9ebf74f1

Allowed Baseline Analysis

During baseline capture (http_allowed.pcapng), client connection requests to http://127.0.0.1/firewall_lab.html completed a standard TCP 3-way handshake:

1. Frame 1: Client sends [SYN] from ephemeral port 56042 to server port 80.
2. Frame 2: Server responds with [SYN, ACK].
3. Frame 3: Client completes handshake with [ACK].
4. Frame 4–6: Client issues GET /firewall_lab.html HTTP/1.1, and server returns HTTP/1.1 200 OK containing the HTML body.

Blocked Traffic Analysis and Counter Correlation

When re-issuing curl -v --connect-timeout 5 http://127.0.0.1/firewall_lab.html under active filtering:

- Client Behavior: curl timed out after 5002ms with curl: (28) Connection timed out.
- Packet Capture (http_blocked.pcapng): Client transmitted an initial [SYN] (Frame 1), followed by repeated [TCP Retransmission] attempts at relative timestamps 1.003s (Frame 2) and 2.028s (Frame 3). No [SYN, ACK] or HTTP payload was generated by the server.
- Rule Counter Correlation: iptables -L INPUT -n -v logged 5 packets / 300 bytes matched and dropped by Rule 1.

DROP versus REJECT Discussion

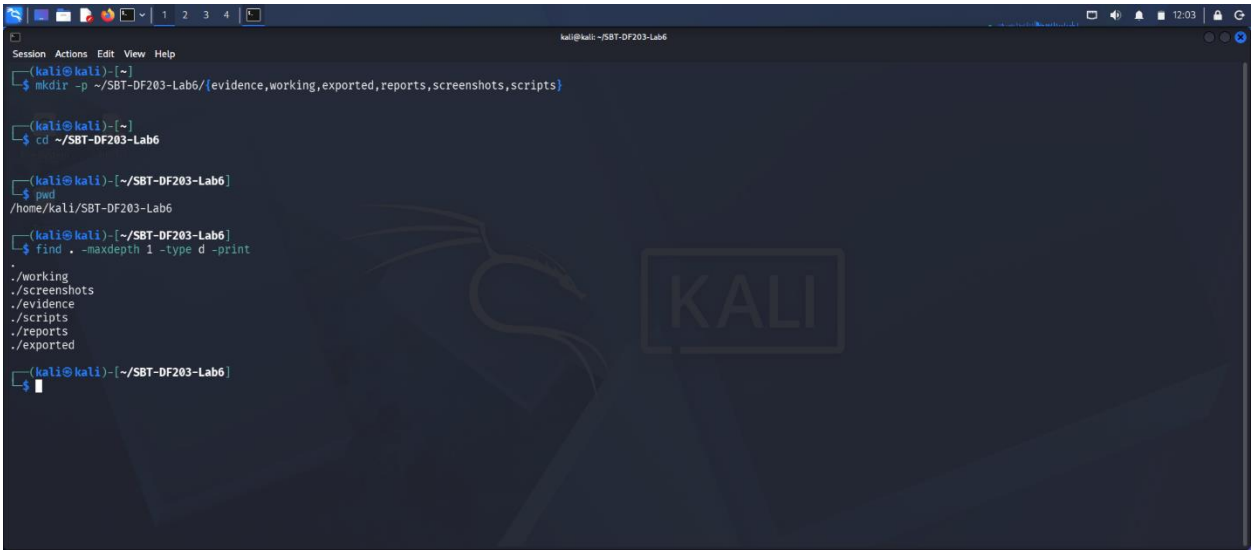
- -j DROP (Observed): Silently discards matching packets. The network stack produces zero response packets. The sender cannot differentiate between firewall blocking and network route loss, causing repeated TCP retransmissions until client application timeout.
- -j REJECT (Theoretical): Actively responds to matching packets by returning an explicit TCP Reset (TCP RST) or ICMP Port Unreachable packet. On packet captures, this displays an immediate [RST, ACK] frame from the server, and the client application fails instantly with Connection refused rather than timing out.

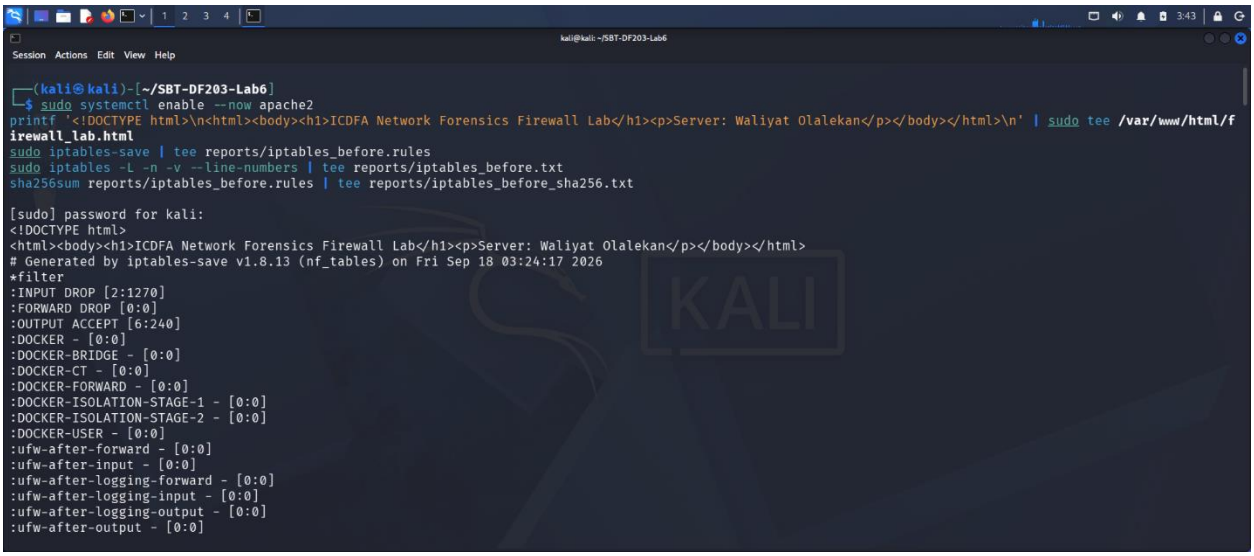
Cleanup and Restored Service

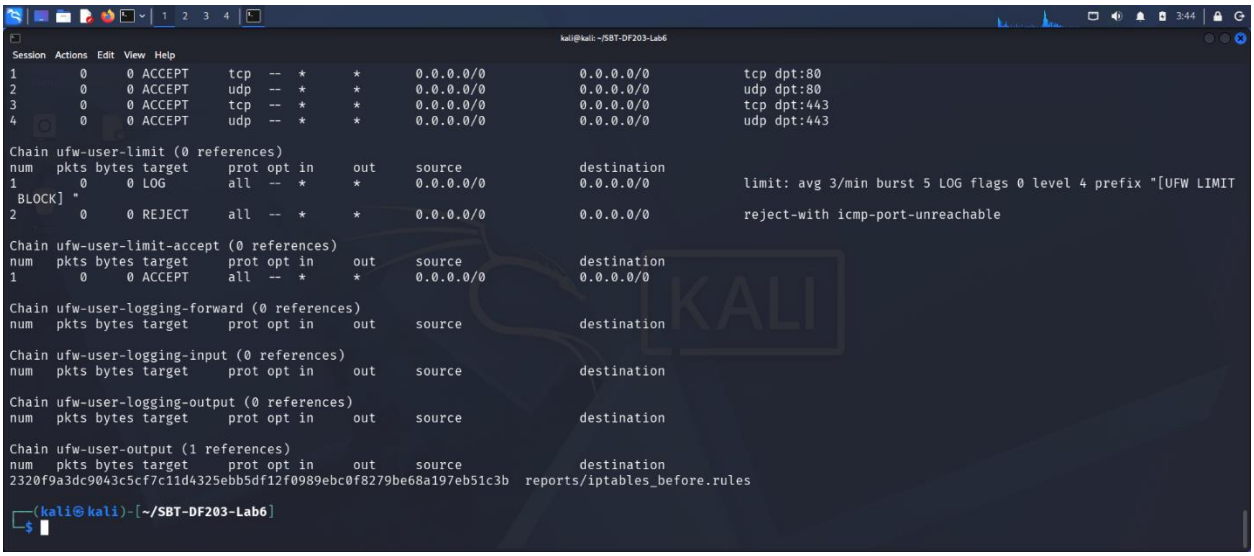
To restore standard web service operations, the rule was deleted and verified: sudo iptables -D INPUT -i lo -p tcp --dport 80 -j DROP.

Subsequent curl requests succeeded with HTTP 200 OK, confirming full-service restoration.

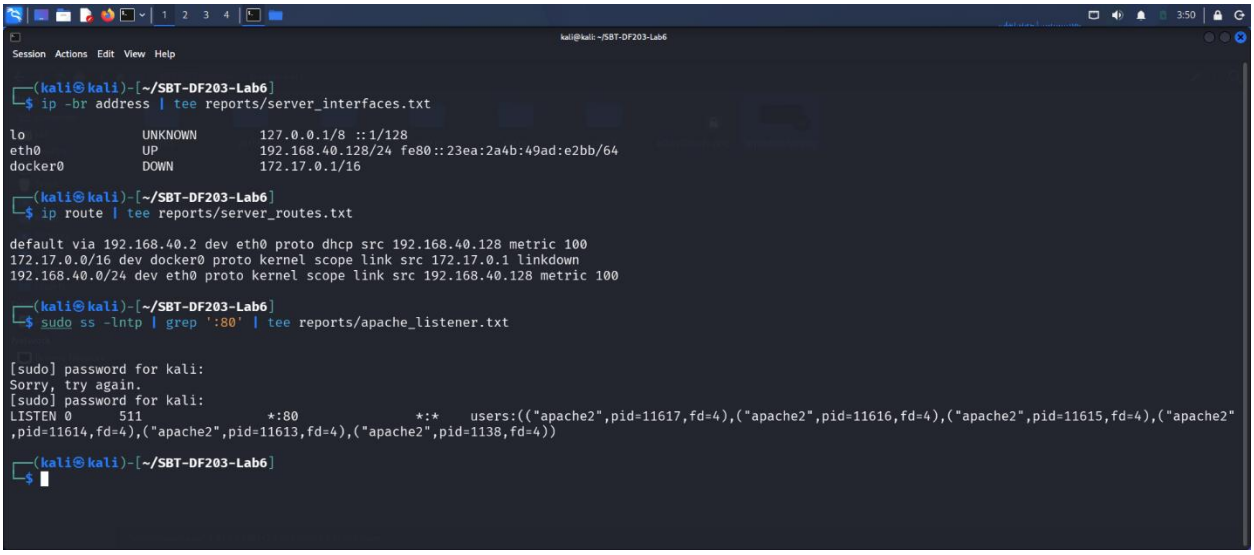
Lab Folder Structure and Evidence Preparation

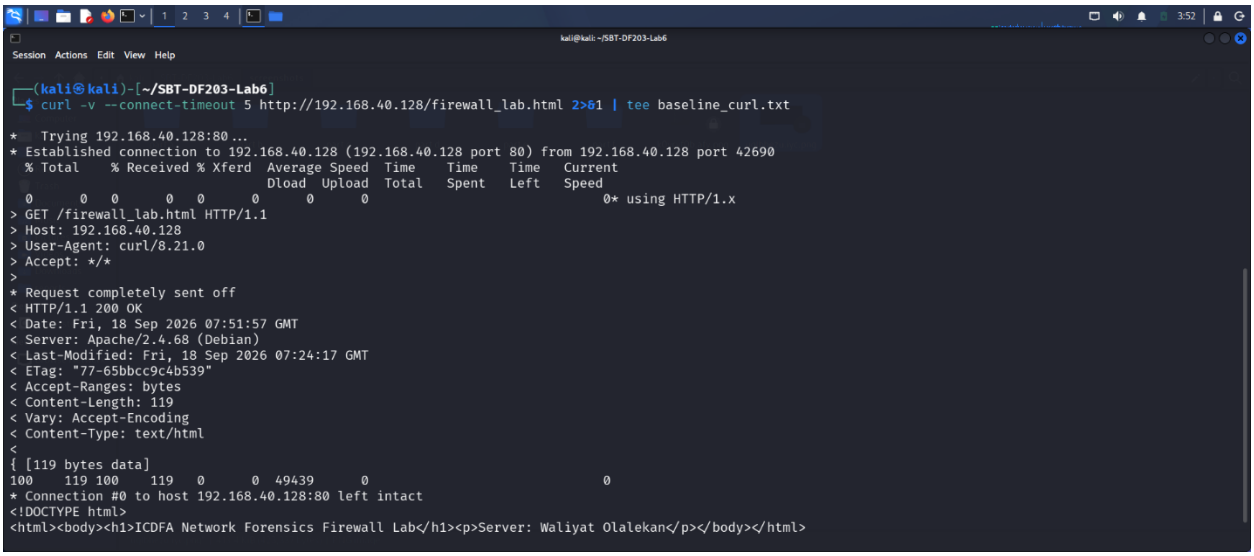

kali@kali: ~/SBT-DF203-Lab6
Session Actions Edit View Help
(kali㉿kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
(kali㉿kali)-[~]
$ cd ~/SBT-DF203-Lab6
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ pwd
/home/kali/SBT-DF203-Lab6
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported
(kali㉿kali)-[~/SBT-DF203-Lab6]
$


kali@kali: ~/SBT-DF203-Lab6
Session Actions Edit View Help
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1> Server: Waliyat Olalekan </body></html>\n' | sudo tee /var/www/html/firewall_lab.html
sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
[sudo] password for kali:
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1> Server: Waliyat Olalekan </body></html>
Generated by iptables-save v1.8.13 (nf_tables) on Fri Sep 18 03:24:17 2026
*filter
:INPUT DROP [2:1270]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [6:240]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
:ufw-after-forward - [0:0]
:ufw-after-input - [0:0]
:ufw-after-logging-forward - [0:0]
:ufw-after-logging-input - [0:0]
:ufw-after-logging-output - [0:0]
:ufw-after-output - [0:0]

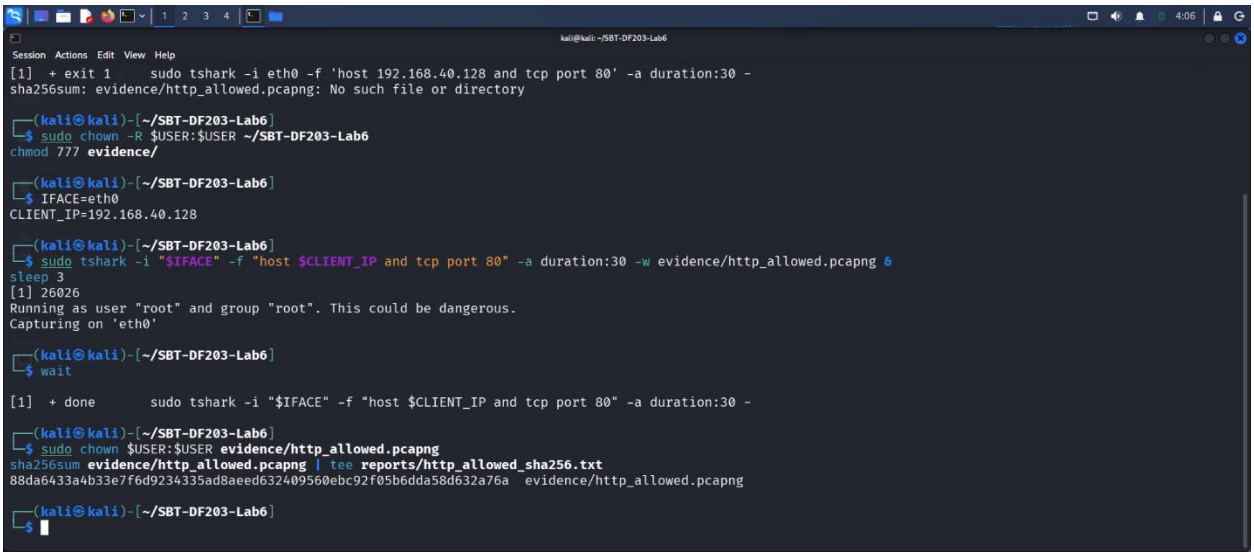

kali@kali: ~/SBT-DF203-Lab6
Session Actions Edit View Help
1 0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
2 0 0 ACCEPT udp -- * * 0.0.0.0/0 0.0.0.0/0 udp dpt:80
3 0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443
4 0 0 ACCEPT udp -- * * 0.0.0.0/0 0.0.0.0/0 udp dpt:443
Chain ufw-user-limit (0 references)
num pkts bytes target prot opt in out source destination
1 0 0 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 limit: avg 3/min burst 5 LOG flags 0 level 4 prefix "[UFW LIMIT BLOCK] "
2 0 0 REJECT all -- * * 0.0.0.0/0 0.0.0.0/0 reject-with icmp-port-unreachable
Chain ufw-user-limit-accept (0 references)
num pkts bytes target prot opt in out source destination
1 0 0 ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0
Chain ufw-user-logging-forward (0 references)
num pkts bytes target prot opt in out source destination
Chain ufw-user-logging-input (0 references)
num pkts bytes target prot opt in out source destination
Chain ufw-user-logging-output (0 references)
num pkts bytes target prot opt in out source destination
Chain ufw-user-output (1 references)
num pkts bytes target prot opt in out source destination
2320f9a3dc9043c5cf7c11d4325ebb5df12f0989ebc0f8279be68a197eb51c3b reports/iptables_before.rules
(kali㉿kali)-[~/SBT-DF203-Lab6]
$

Part A - Record the Lab Network and Baseline Access

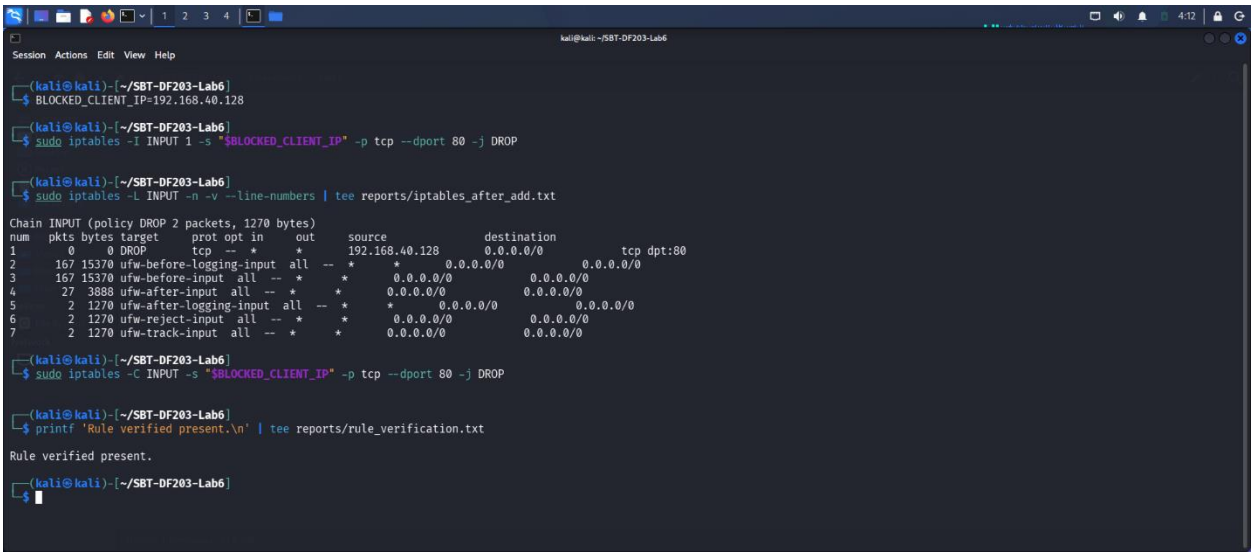




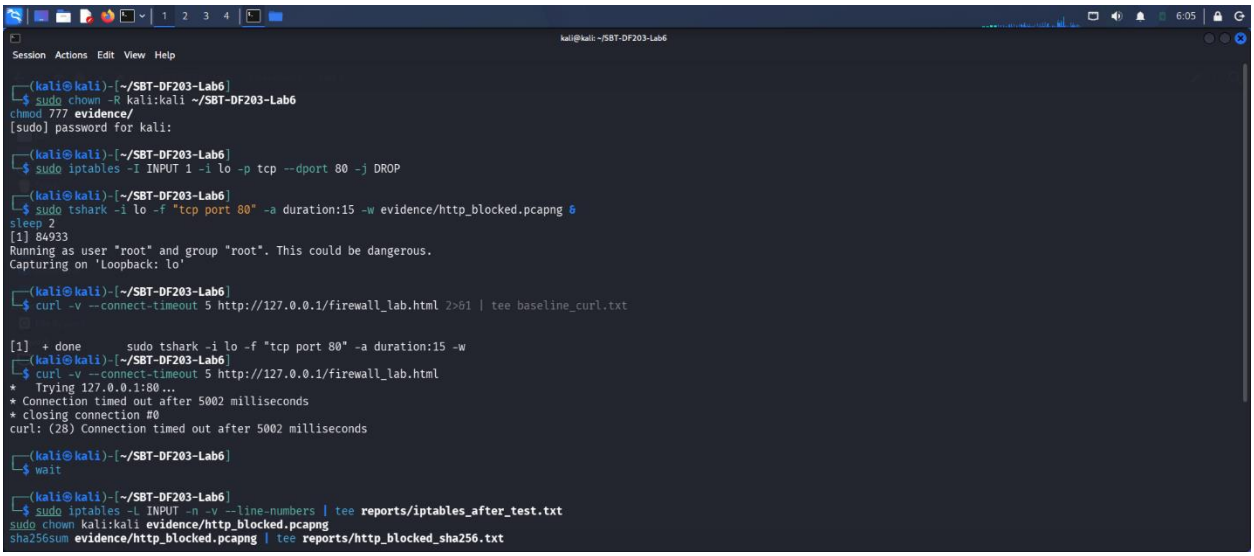
Part B - Capture the Allowed HTTP Baseline

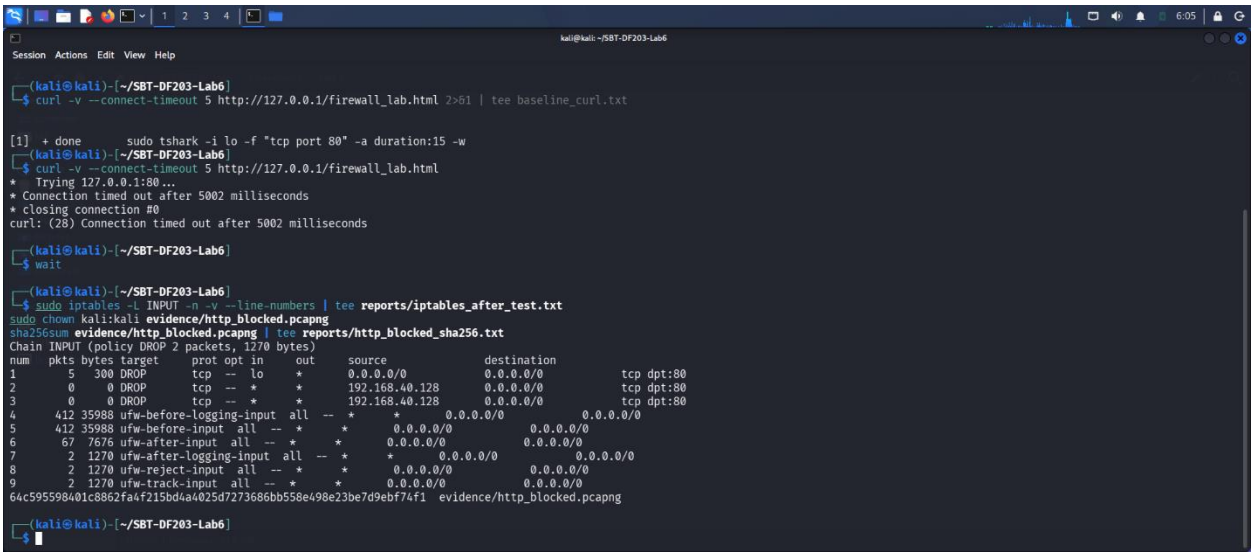


Part C - Apply and Verify the Blocking Rule

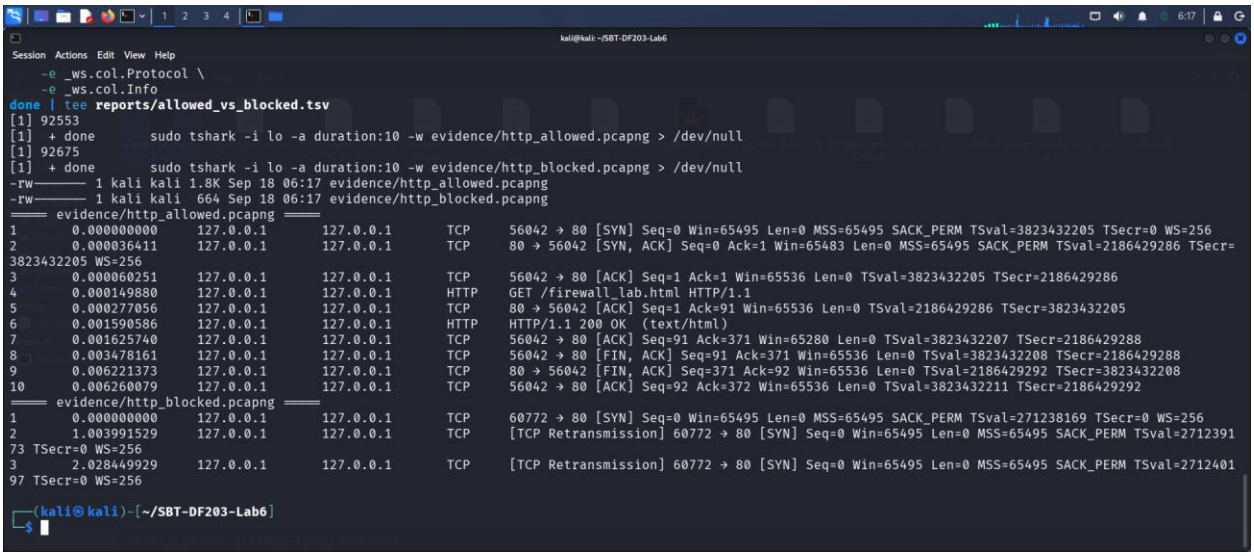


Part D - Capture Blocked Traffic and Rule Counters





Part E - Compare Allowed and Blocked Captures



Indicator	Allowed capture (http_allowed.pcapng)	Blocked capture(http_blocked.pcapng)
Client SYN visible?	Yes (Frame 1: 56042 -> 80 [SYN])	Yes (Frame 1: 60772 -> 80 [SYN])
Server SYN-ACK visible?	Yes (Frame 2: 80 -> 56042 [SYN, ACK])	No (Silently discarded by Netfilter DROP rule)
Handshake completed?	Yes (Frame 3: 56042 -> 80 [ACK])	No (Connection stalled at SYN_SENT)
HTTP GET visible?	Yes (Frame 4: GET /firewall_lab.html HTTP/1.1)	No (Application layer data never transmitted)
HTTP response visible?	Yes (Frame 6: HTTP/1.1 200 OK (text/html))	No (No response generated by server)
Retransmissions/timeouts	None (Orderly teardown with FIN, ACK)	Multiple TCP Retransmissions (Frames 2 & 3: [TCP Retransmission] at ~1s and ~2s)
iptables counter change	No match on custom blocking rules	Rule 1 matched: 5 pkts, 300 bytes dropped
curl result	HTTP/1.1 200 OK (HTML body rendered)	curl: (28) Connection timed out after 5002 milliseconds

Part F - Remove the Rule and Restore Access



Part G - NFQUEUE Observation Guide

1. How NFQUEUE Works: The NFQUEUE iptables target delegates packet filtering and verdict decisions from kernel space to a custom user space application via a Netlink socket. Matching network packets are held in a kernel queue buffer until the user space daemon processes them.

2. User space Callback Decisions: A python application (using NetFilterQueue) receives raw packet payloads in a callback routine. Inside this function, code can inspect headers/payloads and issue one of three verdicts:

    - packet.accept(): Pass packet through netfilter.
    - packet.drop(): Silently discard packet.
    - packet.set_payload(modified_data) + packet.accept(): Modify packet bytes on the fly (e.g., dynamic payload rewriting or inspection).

3. Execution commands:
   # Rule addition passing port 80 traffic to queue 0
   sudo iptables -A INPUT -p tcp --dport 80 -j NFQUEUE --queue-num 0

   # Verify queue rule presence
   sudo iptables -L INPUT -n -v

   # Active python listener process
   python3 nfqueue_listener.py

   # Critical Cleanup Rule
   sudo iptables -D INPUT -p tcp --dport 80 -j NFQUEUE --queue-num 0

4. Operational Hazard Notice: If an NFQUEUE rule remains active in iptables without a corresponding python process attached and listening to --queue-num 0, the kernel queue buffer will quickly fill up. Once full, all matching network traffic will stall completely and drop indefinitely.

Forensic Interpretation Questions

1. Why does DROP commonly cause SYN retransmissions and a timeout?

When an iptables rule applies -j DROP, netfilter silently deletes incoming TCP [SYN] packets at the kernel level before they reach the socket layer. Because no [SYN, ACK] or [RST] response is returned across the wire, the client OS network stack assumes packet loss due to network congestion. Standard TCP stack behavior dictates retransmitting the original [SYN] frame using exponential backoff intervals (e.g., 1s, 2s) until the application's configured connection timeout limit is breached (curl error 28).

2. How would a REJECT rule differ in the packet capture and client output?

- Packet Capture: Rather than showing unanswered [SYN] retransmissions, a REJECT rule causes netfilter to immediately return a TCP Reset ([RST, ACK]) packet (or ICMP Port Unreachable).
- Client Output: Instead of hanging for 5+ seconds before timing out, curl or client applications fail immediately with an explicit curl: (7) Failed to connect to... Connection refused error.

3. Why are firewall counters valuable corroborating evidence?

Firewall packet/byte counters (iptables -L -v) provide objective, host-side kernel proof that specific traffic matched a rule at an exact moment. Correlating an increase in DROP rule packet counts with relative timestamps from a network pcap proves that the target host received the physical frames but intentionally blocked them at the OS level.

4. What evidence would show that the web server was down rather than firewall-blocked?

If the web server daemon (Apache) were down without an active firewall block:

- pcap Evidence: The OS kernel network stack would receive the client [SYN] on port 80 and immediately return an automatic kernel-generated [RST, ACK] packet because no process is bound to the listening socket.
- Listener Logs: ss -lntp | grep ':80' would return no active socket process.
- Firewall Counters: iptables rule counters on custom blocking rules would remain at 0.

5. What is the risk of deleting a rule by line number after other rules have changed?

iptables line numbers are dynamic index positions. If rules above line N are inserted, deleted, or reordered prior to running iptables -D INPUT N, the target line number shifts. Executing a deletion by line number blindly risks deleting an incorrect active security rule, potentially exposing administrative ports or severing critical management access.

Conclusion

The experimental data confirms that host-based firewalls operating at the netfilter level successfully intercept inbound network sessions prior to service socket processing. Forensic identification of -j DROP actions rely on observing TCP SYN retransmission sequences in pcap files combined with non-zero counter progression in system firewall logs.